MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of Faculty: Dr. Neeraj Jain
Subject: Microprocessor & Interfaces (4CS4-02)
Semester: IV **Session: 2023-24 (Even Sem.)**
Branch: CSE **Batch: A**
Unit: 5 **Date of Submission: 5/4/2024**

Lecture No.	Topics Covered	Page No.	Total Page
1	Programming Timer Interrupt	1-4	4
2	Programming External Hardware Interrupt	5-6	2
3	Programming the serial hardware Interrupt	7-9	3
4	Programming the serial communication interrupt	10-12	3
5	Programming 8051 timers & counters	13-18	6

References

1. R. Gaonkar, Microprocessor Architecture Programming, Penram International Publishing

2. Hall D.V., microprocessor & Interfacing-Programming & hardware, TMH, 2008

3. https://www.elprocus.com/8051-microcontroller-8-16-bit-timers-and-counters/

Faculty HOD Dean Academics 05/04/24

UNIT-5 (Programming & Application of 8051)

Programming Timer Interrupt

Lecture No.-1

At any given instant, multiple processes can be running in the 8051. Additionally, a number of external devices can also be connected to it at the same time. So it's safe to say that multiple peripherals or processes could require the CPU at any given moment. As the 8051 has a single-core CPU it can tend to only one task at a given instant of time.

An interrupt is a signal sent by hardware or software processes calling for the immediate attention of the CPU. Once the CPU receives this signal it stops whatever it is doing and takes care of that particular resource.

Types of interrupts in 8051

The 8051 has a total of six interrupts and each interrupt has a designated interrupt service routine (ISR)/interrupt handler assigned to it. The ISR is a predefined code that is stored at a particular memory location in the ROM that the microcontroller executes when the designated interrupt arises.

Interrupt	ROM location of its ISR	Pin	Flag clearing
Reset	0000H	9	Auto
External hardware interrupt 0(INT0)	0003H	P3.2(12)	Auto
Timer 0 interrupt(TF0)	000BH		Auto
External hardware interrupt 1(INT1)	0013H	P3.3(13)	Auto
Timer 1 interrupt(TF1)	001BH		Auto
Serial communication interrupt(RI and TI)	0023H		Programmer clears it

Interrupt Enable register

This register is responsible for enabling and disabling the interrupt. EA register is set to one for enabling interrupts and set to 0 for disabling the interrupts. Its bit sequence and their meanings are shown in the following figure.

The structure of the IE register is given below and all the bits are described thereafter.

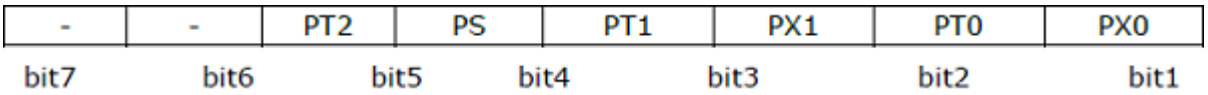
EA	-	-	ES	ET1	EX1	ET0	EX0

EA	IE.7	It disables all interrupts. When EA = 0 no interrupt will be acknowledged and EA = 1 enables the interrupt individually.
-	IE.6	Reserved for future use.
-	IE.5	Reserved for future use.
ES	IE.4	Enables/disables serial port interrupt.
ET1	IE.3	Enables/disables timer1 overflow interrupt.
EX1	IE.2	Enables/disables external interrupt1.
ET0	IE.1	Enables/disables timer0 overflow interrupt.
EX0	IE.0	Enables/disables external interrupt0.

IP (Interrupt Priority) Register

We can change the priority levels of the interrupts by changing the corresponding bit in the Interrupt Priority (IP) register as shown in the following Figure.

- A low priority interrupt can only be interrupted by the high priority interrupt, but not interrupted by another low priority interrupt.
- If two interrupts of different priority levels are received simultaneously, the request of higher priority level is served.
- If the requests of the same priority levels are received simultaneously, then the internal polling sequence determines which request is to be serviced.



-	IP.6	Reserved for future use.
-	IP.5	Reserved for future use.
PS	IP.4	It defines the serial port interrupt priority level.
PT1	IP.3	It defines the timer interrupt of 1 priority.
PX1	IP.2	It defines the external interrupt priority level.
PT0	IP.1	It defines the timer0 interrupt priority level.
PX0	IP.0	It defines the external interrupt of 0 priority level.

TCON Register: In addition to the above two registers, the TCON register specifies the type of external interrupt to the 8051 microcontroller. The two external interrupts, whether edge or level triggered, specify by this register by a set, or cleared by appropriate bits in it. And, it is also a bit addressable register.

Timer 0 and Timer 1 interrupts are generated by the timer register bits TF0 and TF1.:

- **Selecting the timer by configuring TMOD register and its mode of operation.**
- **Choosing and loading the initial values of TLx and THx for appropriate modes.**
- **Enabling the IE registers and corresponding timer bit in it.**
- **Setting the timer run bit to start the timer.**
- **Writing the subroutine for the timer for time required and clear timer value TRx at the end of subroutine.**

```
// Timer0 mode1 for blinking LED using interrupt method
#include<reg51.h>
sbit Blink Led = P2^0;    //LED connected to port 2 Zeroth pin
void timer0_ISR (void) interrupt 1    //interrupt no. 1 for Timer 0
{
        Blink Led=~Blink Led;        //Blink LED on interrupt
        TH0=0xFC;              // loading initial values to timer
        TL0=0x66;
}
void main()
{
        TMOD = 0x01;          // mode 1 of Timer0
        TH0 = 0xFC;           // initial values loaded to timer
        TL0 = 0x66;
        ET0 = 1;              // enable timer 0 interrupt
        TR0 = 1;              //start timer
        while(1);             // do nothing
}
```

UNIT-5 (Programming & Application of 8051)

Programming External Hardware Interrupt

Lecture No.-2

The 8051 microcontrollers are able to respond to external events through its external interrupts, INT0 and INT1.

programming external interrupts (Level and edge triggering)

The 8051 has two external hardware interrupts. Pin 12 (P3.2) and pin 13 (P3.3) of the 8051, designated as INTO and INT1, are used as external hardware interrupts. Upon activation of these pins, the 8051 gets interrupted in whatever it is doing and jumps to the vector table to perform the interrupt service routine. In this section we study these two external hardware interrupts of the 8051 with some examples

External Interrupt 0 (INT0)

- It is connected to the 8051's pin PORT3.2.
- An interrupt request is issued when this pin transitions from low to high in response to an external signal.
- It is possible to program the microcontroller to carry out a particular Interrupt Service Routine (ISR) in response to this interrupt.
- Set the IE (Interrupt Enable) bit for INT0 in the TCON register and configure the IT0 (Interrupt Type 0) bit in the TCON register corresponding to the desired triggering condition (edge or level-triggered) in order to enable and configure INT0.

External Interrupt 1 (INT1)

- It is connected to the 8051's pin PORT3.3
- When that particular pin encounters a low-to-high transitions, INT1, like INT0, creates an interrupt request.

- By configuring the IT1 (Interrupt Type 1) bit in the TCON register and setting the IE bit for INT1 in the TCON register, one can enable and configure INT1.
- A specific ISR can be executed by the microcontroller in response to INT1.

```
/Edge trigger external interrupt
void main()
{
IT0 = 1;   //Configure interrupt 0 for falling
             edge on INT0
EX0 = 1;   //Enable EX0 Interrupt
EA  = 1;   //Enable Global Interrupt Flag
}
void ISR_ex0(void) interrupt 0
{
        <body of interrupt>
}
```

UNIT-5 (Programming & Application of 8051)

Programming the serial hardware Interrupt

Lecture No.-3

To use the serial interrupt the ES bit along with the EA bit is set. Whenever one byte of data is sent or received, the serial interrupt is generated and the TI or RI flag goes high. Here, the TI or RI flag needs to be cleared explicitly in the interrupt routine (written for the Serial Interrupt).

The programming of the Serial Interrupt involves the following steps:

1. Enable the Serial Interrupt (configure the IE register).
2. Configure SCON register.
3. Write routine or function for the Serial Interrupt. The interrupt number is 4.
4. Clear the RI or TI flag within the routine.

Example code

Send 'A' from serial port with the use of interrupt

```
// Sending 'A' through serial port with interrupt

// XTAL frequency 11.0592MHz

void main()

{

        TMOD = 0x20;

        TH1 = -1;

        SCON = 0x50;

        TR1 = 1;
```

```
        IE = 0x90;
        while(1);
}
void ISR_sc(void) interrupt 4
{
       if(TI==1)
       {
              SBUF = 'A';
              TI = 0;
       }
       else
              RI = 0;
```

Example code

```
// Receive data from serial port through interrupt
// XTAL frequency 11.0592MHz
void main()
{
       TMOD = 0x20;
       TH1 = -1;
       SCON = 0x50;
       TR1 = 1;
       IE = 0x90;
       while(1);
}
void ISR_sc(void) interrupt 4
{
```

```
        unsigned char val;
        if(TI==1)
        {
                TI = 0;
        }
        else
        {
                val = SBUF;
                RI = 0;
        }
}
```

UNIT-5 (Programming & Application of 8051)

Programming the serial communication interrupt

Lecture No.-4

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication protocol used with 8051 microcontrollers. Data is sent over a single cable, bit by bit, in serial transmission. In this sense, "interrupts" refers to the processes that enable the microcontroller to react quickly to external events.

Addressing UART communication with the 8051's interrupts:

- **Initialization of UART-** Set the data format, baud rate, and enable the UART module by configuring the UART registers.
- **Interrupt Enable**– Depending on the operation you wish to interrupt for, enable the UART's transmit interrupt (TI) or receive interrupt (RI).
- **ISR (interrupt service routine)-** To handle the interrupt, write an ISR. The ISR in UART communication normally verifies whether the transmit buffer is ready (TI) or whether data has been received (RI).
- **Clearing the Flag- To** recognize the interrupt and get ready for the next one, in the ISR, clear the associated interrupt flag (RI or TI).

Interrupt Vector Table

The addresses of different interrupt service routines (ISRs) are stored in a table called the Interrupt Vector Table (IVT) in an 8051 microcontroller. It is a vital aspect of the interrupt handling mechanism in the microcontroller. When an interrupt occurs the interrupt specific ISR is executed by jumping the program counter to the corresponding address in the IVT. There are memory areas set aside specifically for the IVT in the 8051 microprocessors. Every interrupt has a specific place in the IVT, and the addresses kept there point to the program memory's associated ISR's start. By guiding the program flow to the proper place, the IVT enables the microcontroller to respond to external events like hardware interrupts or external signals quickly and effectively.

Interrupt	Flags	Interrupt Vector Address
Reset	–	0000H
INT0 (External Interrupt 0)	IE0	0003H
Timer0	TF0	000BH
INT1 (External Interrupt 1)	IE1	0013H
Timer 1	TF1	001BH
Serial Interrupt	TI/RI	0023H

Advantages of 8051 Microcontroller Interrupt

- By quickly resolving problems, interrupts can be used for error detection and handling, improving the system's robustness and reliability.
- The 8051's capability to operate in a low-power mode and wake up only in response to an interrupt helps battery-powered applications operate more energy-efficiently.
- Interrupts make it possible to break up large, complicated processes into smaller, more manageable subroutines, which improve the readability and organization of the code.
- By allowing the microcontroller to be interrupted in response to a predetermined event, the interrupt system minimizes CPU overhead and boosts overall system performance. This eliminates the requirement for constant input polling.
- A few 8051 variations enable interrupt priority levels, which give important tasks the upper hand over less important ones and improve responsiveness and control of the system.

Disadvantages of 8051 Microcontroller Interrupt

- The number of interrupt sources that can be supported is limited by the 8051 microcontroller's two external interrupt pins (INT0 and INT1).
- The 8051's interrupt priority cannot be dynamically adjusted; it is fixed. This could render handling several interruption sources with different priorities difficult.
- Variations in the microcontroller's response time to an interrupt might affect real-time applications where reliable and consistent response times are essential.
- Careful programming is necessary to prevent unexpected behavior while handling interrupts, particularly when working with nested interruptions or crucial portions.
- Since the 8051 lacks vectored interrupts, unlike some other modern microcontrollers, it is required to manually examine interrupt flags in order to identify the interrupt's source.

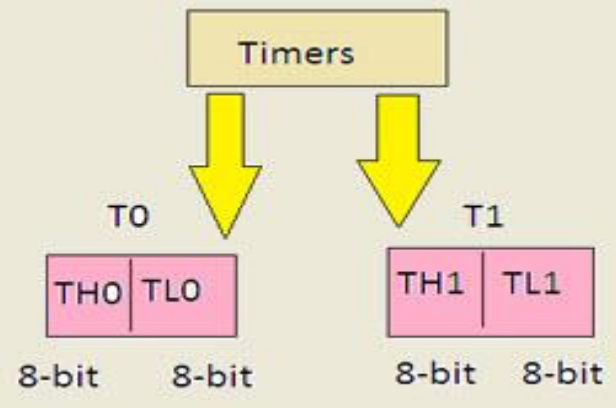
UNIT-5 (Programming & Application of 8051)

Programming 8051 Timers & Counters

Lecture No.-5

Many of the microcontroller applications require counting of external events such as frequency of the pulse trains and generation of precise internal time delays between computer actions. Both these tasks can be implemented by software techniques, but software loops for counting, and timing will not give the exact result rather more important functions are not done. To avoid these problems, timers and counters in the micro-controllers are better options for simple and low-cost applications. These timers and counters are used as interrupts in 8051 microcontroller.

There are two 16-bit timers and counters in 8051 microcontroller: timer 0 and timer 1. Both timers consist of 16-bit register in which the lower byte is stored in TL and the higher byte is stored in TH. Timer can be used as a counter as well as for timing operation that depends on the source of clock pulses to counters.



Counters and Timers in 8051 microcontroller contain two special function registers: TMOD (Timer Mode Register) and TCON (Timer Control Register), which are used for activating and configuring timers and counters.

Timer Mode Control (TMOD): TMOD is an 8-bit register used for selecting timer or counter and mode of timers. Lower 4-bits are used for control operation of timer 0 or counter0,

and remaining 4-bits are used for control operation of timer1 or counter1.This register is present in SFR register, the address for SFR register is 89th.

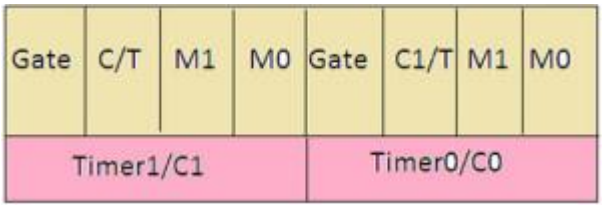


Fig: **Timer Mode Control (TMOD)**

Gate: If the gate bit is set to '0', then we can start and stop the "software" timer in the same way. If the gate is set to '1', then we can perform hardware timer.

C/T: If the C/T bit is '1', then it is acting as a counter mode, and similarly when set C+=/T bit is '0'; it is acting as a timer mode.

Mode select bits: The M1 and M0 are mode select bits, which are used to select the timer operations. There are four modes to operate the timers.

Mode 0: This is a 13-bit mode that means the timer operation completes with "8192" pulses.

Mode 1: This is a16-bit mode, which means the timer operation completes with maximum clock pulses that "65535".

Mode 2: This mode is an 8-bit auto reload mode, which means the timer operation completes with only "256" clock pulses.

Mode 3: This mode is a split-timer mode, which means the loading values in T0 and automatically starts the T1.

M0	M1	Mode	Timer Pulses
0	0	M0	13-bit-2^13-8192
0	1	M1	16-bit-2^16-65535 pulses
1	0	M2	8-bit-autoreload mode-2^8= 256 pulses
1	1	M3	Split mode(load the values in T0 automatically start the T1

Mode selection Values of timers and counter in 8051

T/C	M0	M1	M2
T0	00h	0h	02h
T1	00h	10h	20h
C0	04h	05h	06h
C1	40h	50h	60h

Timer Control Register (TCON): TCON is another register used to control operations of counter and timers in microcontrollers. It is an 8-bit register wherein four upper bits are responsible for timers and counters and lower bits are responsible for interrupts.

TF1: The TF1 stands for 'timer1' flag bit. Whenever calculating the time-delay in timer1, the TH1 and TL1 reaches to the maximum value that is "FFFF" automatically.

EX: while (TF1==1)

Whenever the TF1=1, then clear the flag bit and stop the timer.

TR1: The TR1 stands for timer1 start or stop bit. This timer starting can be through software instruction or through hardware method.

EX: gate=0 (start timer 1 through software instruction)
TR1=1; (Start timer)

TF0: The TF0 stands for 'timer0' flag-bit. Whenever calculating the time delay in timer1, the TH0 and TL0 reaches to a maximum value that is 'FFFF', automatically.

EX: while (TF0==1)
Whenever the TF0=1, then clear the flag bit and stop the timer.

TR0: The TR0 stands for 'timer0' start or stop bit; this timer starting can be through software instruction or through hardware method.

EX: gate=0 (start timer 1 through software instruction)
TR0=1; (Start timer)

Time Delay Calculations for 8051 Microcontroller

The 8051 microcontroller works with 11.0592 MHz frequency.

Frequency 11.0592MHz=12 pules

1 clock pulse =11.0592MHz/12

F =0.921 MHz

Time delay=1/F

T=1/0.92MHz

T=1.080506 us (for '1' cycle)

1000us=1MS

1000 ms = 1 sec

WAP to generate 100 ms time delay using timer 0 mode 0

```
#include<reg51.h>
void main()
{                              100ms/1.08592us
                               =920878/20=4604  (20 for loop)
                               8191-4604=3587
 unsigned char i;              3587 convetted by hex =0E03

 TMOD=0x00;
for(=0;i<20;i++)
{
 TL0=0x03;
TH0=0x0E;
TR0=1;
while(TF==0);
TF=0;
}
TR0=0;
}
```

Counters in 8051

We can use a counter by keeping C/T bit high, i.e., logic '1' in the TMOD register. For better understanding, we have given one program which uses timer 1 as a counter. Here the LEDs are connected to 8051 Port 2, and the switch to the timer1 pin P3.5; and therefore, if the switch is pressed, the value will be counted. Otherwise, an externally connected sensor to this counter pin as input does this counting operation.

WAP to timer 1 used as a counter

```
#incluede<reg51.h>
sbit clock=P3^5;
void main()
{

  unsigned char i=0;
  TMOD=0x60;                   selected counter
  TH1=0x00;                    maximum plulse value
  TL1=0x00;
  clock=1;                      p3.5 is high
  while(1)
 {
  TR1=1;
 count : a=TL1;
  P2=a;                         send the values on port 2
 if(TF1==0) goto count;
}
TR1=0;
TF1=0;
}
```

Application of Timers and Counters in 8051

Time delay circuit Using 8051 microcontroller

The below Fig. shows how the timer operation can be implemented for switching the LEDs in an effective way. The time delay operation for the set of LEDs is programmed in a microcontroller in the manner discussed above. Here, a set of LEDs are connected to the port 2 with a common supply system. When this circuit is turned on based on the time delay program in the microcontroller appropriately, these LEDs are switched on.

